HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

PROJECT 1 REPORT

A RAG-ENHANCED SYSTEM FOR JOB DESCRIPTION AUTHORING AND INTERVIEW QUESTION GENERATION

Supervisor: Dr. Tran Viet Trung

Class ID: 761977

Course Code: IT3910E

Course: Project 1

Author: Nguyen Hoang Quan

Student ID: 202416738

Major: Data Science & Artificial Intelligence

Hanoi, 06/2026

DECLARATION

I, NGUYEN HOANG QUAN, student ID 202416738, hereby declare that this PROJECT 1 report entitled “A RAG-ENHANCED SYSTEM FOR JOB DESCRIPTION AUTHORING AND INTERVIEW QUESTION GENERATION” is the result of my own study, system design, implementation, and evaluation under the supervision of Dr. Tran Viet Trung. The system metrics, test results, and implementation descriptions reported in this document were collected from the developed prototype at the time of writing. External algorithms, technical documents, libraries, and official specifications used in the project are cited in the References section. I do not submit copied work as my own result, and I take full responsibility for the integrity of this report.

Hanoi, day … month … year 2026

Student

NGUYEN HOANG QUAN

ABSTRACT

Recruitment teams need consistent job descriptions, explicit evaluation criteria, and role-specific interview questions. In practice, job description authoring is often fragmented across old templates, manual notes, and inconsistent role taxonomies. This makes the process time-consuming, difficult to standardize, and hard to trace. Large language models can accelerate drafting, but using them without enterprise context may produce generic or unsupported content. This report presents SmartHire Composer, a web-based system for creating, editing, versioning, exporting, and improving job descriptions, as well as generating interview questions. The system applies Retrieval-Augmented Generation (RAG) to ground AI-assisted writing in a curated corpus of public job postings.

The proposed solution consists of a React/Vite frontend, a FastAPI backend, PostgreSQL for transactional data and full-text retrieval, Milvus for vector search, and Gemini models for text generation and embeddings. Public ATS postings are normalized with provenance metadata, segmented into overlapping chunks, and embedded into 768-dimensional vectors. At query time, dense cosine retrieval and PostgreSQL lexical retrieval are fused using Reciprocal Rank Fusion, followed by diversity-aware selection to reduce repeated chunks from the same job description. The system also includes JWT authentication, role-based access control, version history, PDF/DOCX export, and graceful fallback when the LLM service is unavailable.

The experimental corpus contains 75 public job descriptions from three companies and 589 indexed chunks. On a 15-query self-retrieval benchmark, the hybrid retrieval pipeline achieved Recall@5 of 1.000, Mean Reciprocal Rank of 0.9556, and a unique-document ratio of 1.000. Five automated backend tests passed, the frontend production build completed successfully, and npm audit reported no known vulnerabilities. These results demonstrate that the proposed architecture is feasible for Project 1, while also identifying the need for expert-labeled relevance evaluation, larger-scale testing, production monitoring, and deeper integration with recruitment workflows in future work.

Keywords: RAG, job description, recruitment, semantic search, hybrid retrieval, Gemini, Milvus, FastAPI.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF EQUATIONS

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
ECR	Elastic Container Registry
ECS	Elastic Container Service
EKS	Elastic Kubernetes Service
FTS	Full-Text Search
HNSW	Hierarchical Navigable Small World
JD	Job Description
JWT	JSON Web Token
LLM	Large Language Model
MMR	Maximal Marginal Relevance
MRR	Mean Reciprocal Rank
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
RRF	Reciprocal Rank Fusion
UI	User Interface
UX	User Experience

CHAPTER 1. INTRODUCTION

1.1 Problem Statement

A job description is a central artifact in the recruitment process. It communicates the hiring need, defines responsibilities and requirements, guides candidate screening, and provides the basis for building interview questions. However, in many organizations, job descriptions are written from scattered templates, old postings, and the personal experience of recruiters or hiring managers. This practice causes three major issues. First, the structure and terminology of job descriptions are often inconsistent across departments. Second, recruiters may find it difficult to reuse relevant historical examples from a large internal library. Third, when a large language model is used without appropriate context, the generated content may be too generic or may contain unsupported claims.

Retrieval-Augmented Generation (RAG) addresses part of this problem by combining information retrieval with text generation. Instead of asking an LLM to generate an answer only from its model parameters, the system first retrieves relevant passages from an external knowledge base and then injects them into the prompt. In this project, the retrieved knowledge consists of real job-description chunks with source metadata. RAG does not eliminate all LLM errors, but it improves domain grounding, enables source traceability, and separates knowledge updates from model training [1].

1.2 Objectives and Scope

The goal of this project is to build a web-based prototype that supports the core lifecycle of a job description. The specific objectives are: (i) collecting and normalizing public job postings with provenance information; (ii) supporting both semantic and keyword search over the job-description corpus; (iii) using retrieved context to support job-description generation and improvement; (iv) generating interview questions based on position, seniority, and competency areas; (v) managing users, access control, and job-description version history; and (vi) quantitatively evaluating retrieval quality.

The scope of Project 1 focuses on a working prototype in a development environment. The experimental corpus contains 75 public job descriptions collected from Cloudflare, Datadog, and Figma. The system does not process real candidate profiles, does not make hiring decisions, and does not collect personal information from application forms. Public ATS APIs and openly accessible job-posting metadata are prioritized. Sources that require authentication, block automated access, or disallow crawling are outside the scope of data collection.

1.3 Solution Direction

The proposed solution follows a multi-layer client-server architecture. The React frontend provides the user interface for composing job descriptions, previewing Markdown content, viewing versions, retrieving similar job descriptions, and exporting documents. The FastAPI backend provides REST APIs and coordinates business logic. PostgreSQL stores transactional data and supports lexical full-text search. Milvus stores dense embedding vectors for semantic search. Gemini models are used for both text generation and embeddings.

The retrieval pipeline combines dense retrieval with lexical retrieval. Dense retrieval captures semantic similarity, while lexical retrieval preserves exact keyword matching for skills, tool names, job titles, and rare expressions. The two ranked lists are fused using Reciprocal Rank Fusion (RRF), and a diversity-aware selection step limits repeated chunks from the same job description. This design makes the system more robust than a pure vector search or pure keyword search approach.

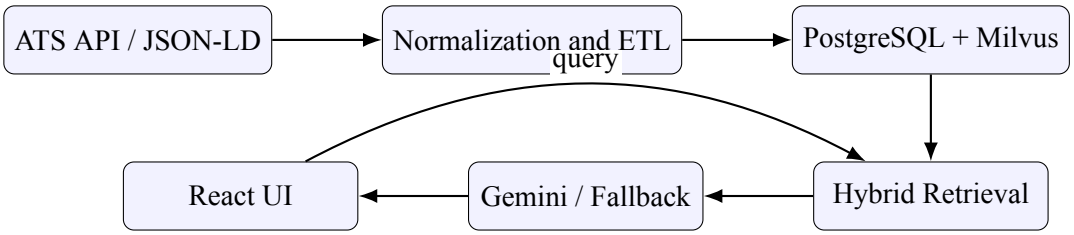


Figure 1.1. High-level processing direction of the proposed system

1.4 Main Contributions

This project contributes a prototype that integrates several components that are often handled separately: real job-posting ingestion with provenance, versioned job-description management, hybrid RAG, AI-assisted writing, interview-question generation, role-based access control, and a usable web interface. The main technical contribution is a retrieval architecture that keeps PostgreSQL and Milvus synchronized through stable chunk identifiers, records an embedding/index manifest, combines dense and lexical ranking through RRF, and applies a per-document diversity constraint. As a result, the system is not merely an LLM wrapper; it is a traceable data-to-retrieval-to-generation pipeline that can be tested and reproduced.

1.5 Report Structure

The remainder of this report is organized as follows. Chapter 2 analyzes the current situation, user roles, functional requirements, and non-functional requirements. Chapter 3 presents the theoretical background and technologies used in the project. Chapter 4 describes system design, data modeling, the RAG pipeline, and implementation details. Chapter 5 reports the ex-

perimental setup, retrieval evaluation, software testing, and discussion of limitations. Chapter 6 concludes the report and proposes future development directions.

CHAPTER 2. REQUIREMENT ANALYSIS

2.1 Current Situation

Traditional document editors can store job-description templates, but they do not provide semantic retrieval over a growing job-description library. Standalone LLM tools can generate content quickly, but they usually lack enterprise-specific context, provenance tracking, version control, and access control. Keyword-based search systems are reliable when the query terms exactly match the stored documents, but they may fail when users describe a role in different words. These observations motivate a combined approach: a transactional database for business data, lexical retrieval for rare and exact terms, dense retrieval for semantic similarity, and LLM generation only after relevant context has been retrieved.

2.2 Actors and User Roles

The system supports four conceptual roles. The *Admin* manages users, roles, and all system-level functions. The *Recruiter* creates job descriptions, improves content, generates interview questions, saves versions, and exports documents. The *Hiring Manager* reviews job descriptions, checks version history, and uses retrieval results to compare similar roles. The *Viewer* role is intended for read-only access in future extensions.

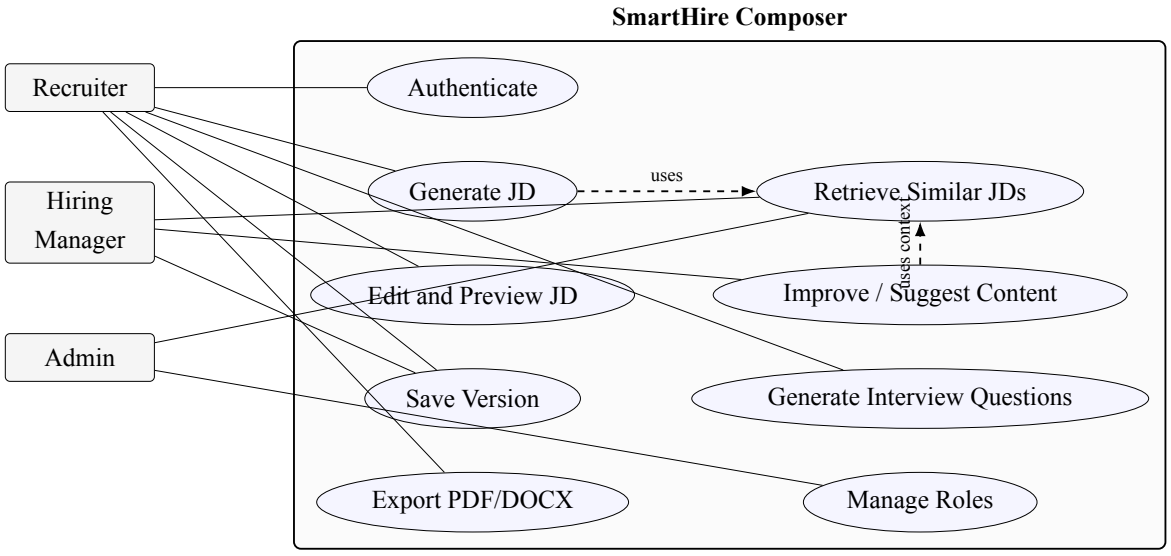


Figure 2.1. UML use case diagram for the main system functions

2.3 Functional Requirements

Table 2.1. Functional requirements of the system

ID	Function	Description
F01	Authentication	Users log in with username and password, receive a JWT, and retrieve the current profile.
F02	JD authoring	Users enter title, department, level, job family, and language to create a draft.
F03	Version management	Each saved edit is recorded with version number, author, timestamp, and change summary.
F04	AI improvement and suggestions	The system improves the whole JD or suggests content for a selected section.
F05	Interview-question generation	The system generates technical, behavioral, and situational questions with competency, difficulty, and rubric.
F06	Retrieval	Users search in dense, lexical, or hybrid mode; filter by company or department; and receive snippet and source URL.
F07	Export	The current JD can be exported to PDF or DOCX.
F08	Data ingestion	The system collects job postings from Greenhouse, Lever, or JSON-LD sources and stores provenance.
F09	Indexing	The system chunks documents, generates embeddings, writes vectors to Milvus, and stores a catalog in PostgreSQL.
F10	Monitoring	Liveness, readiness, and request identifiers are provided for debugging and operations.

2.4 Main Business Workflow

The main workflow begins when a recruiter logs in and enters metadata for a position. The backend builds a query from the title, level, department, and job family. The hybrid retriever returns relevant chunks with company, title, and source URL. The LLM orchestrator wraps external text inside an `UNTRUSTED_SOURCE` block, instructs the model not to execute instructions from that source, and generates a grounded draft. The job description and its first version are stored. The user can then edit the Markdown content, save a new version, generate suggestions, improve the text, and export the result.

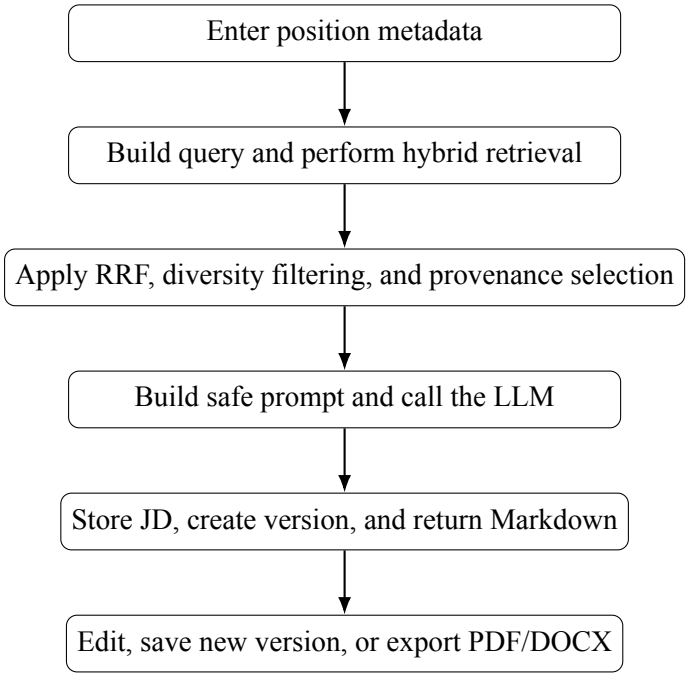


Figure 2.2. Workflow for generating and finalizing a job description

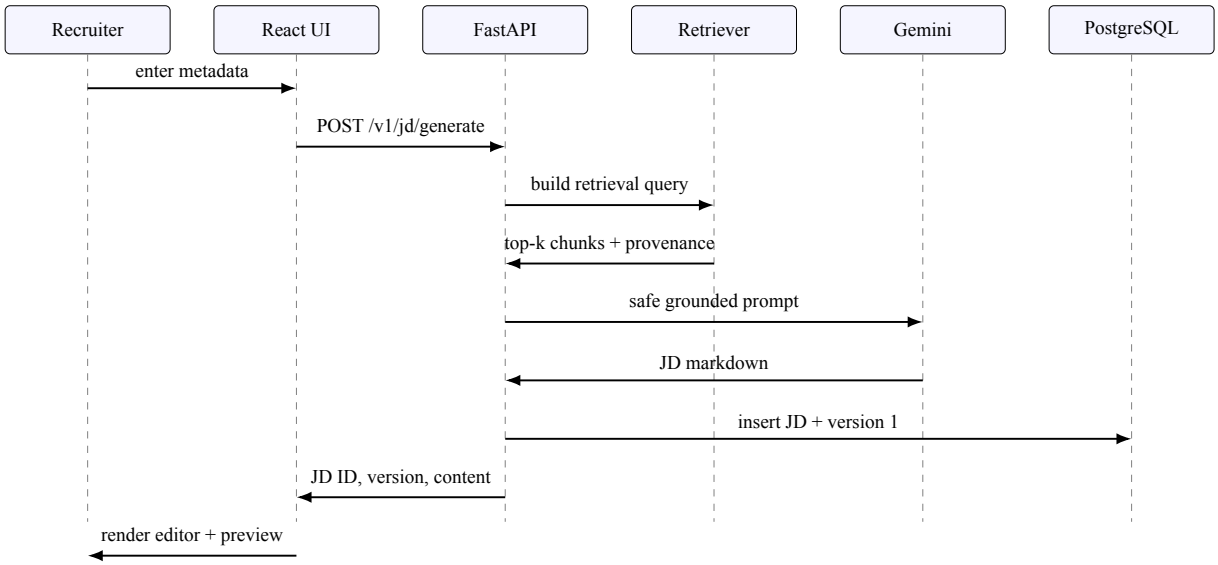


Figure 2.3. UML sequence diagram for generating a job description with RAG

2.5 Use Case Specification

This section specifies the most important use cases. Detailed UI screenshots and API descriptions are provided in later chapters and appendices.

Table 2.2. Use case UC01: User login

Field	Description
Use case ID	UC01

Use case name	User login
Actor	Admin, Recruiter, Hiring Manager, Viewer
Precondition	The user account exists and is active.
Main flow	The user opens the login page, enters username and password, submits the form, and the backend validates the credentials. If valid, the backend returns a JWT and the frontend stores it for subsequent API requests.
Alternative flow	If credentials are invalid, the system displays an error message. If the account is inactive, access is denied.
Postcondition	The user is authenticated and can access functions according to assigned roles.

Table 2.3. Use case UC02: Generate a job description

Field	**Description**
Use case ID	UC02
Use case name	Generate a job description
Actor	Recruiter, Admin
Precondition	The user is authenticated and has permission to create JDs.
Main flow	The user enters job metadata, selects the output language, and requests generation. The backend optionally retrieves similar JD chunks, builds a prompt, calls the LLM, stores the generated content, creates version 1, and returns the Markdown content.
Alternative flow	If the LLM is unavailable, the system returns a minimal deterministic draft or an error message depending on the configured fallback mode.
Postcondition	A new JD exists in the database with its first version recorded.

Table 2.4. Use case UC03: Retrieve similar job descriptions

Field	**Description**
Use case ID	UC03
Use case name	Retrieve similar job descriptions
Actor	Recruiter, Hiring Manager, Admin
Precondition	The RAG index contains chunks and the user is authenticated.
Main flow	The user enters a query and selects retrieval mode. The backend performs dense, lexical, or hybrid retrieval, applies filters and diversity constraints, and returns ranked results with snippets and source metadata.

Alternative flow	If dense embedding fails in hybrid mode, the system falls back to lexical retrieval. If no result is found, the frontend displays an empty-state message.
Postcondition	The user can inspect relevant examples and use them as context for authoring.

2.6 Non-functional Requirements

Table 2.5. Non-functional requirements

Category	Requirement
Security	Passwords are hashed with bcrypt; APIs are protected by JWT and RBAC; secrets are not committed to source code; CORS is configured with an allow-list.
Reliability	The backend should still start when Milvus is temporarily unavailable; hybrid retrieval should degrade to lexical retrieval when embeddings fail.
Traceability	Each JD has source ID, source URL, company, observed timestamp, and SHA-256 hash; each request has an X-Request-ID.
Performance	HNSW is used for vector search; GIN is used for full-text search; query embeddings are cached; candidate and top-k sizes are bounded.
Maintainability	The codebase is modularized into API, service, CRUD, schema, and utility layers; migrations are ordered; automated tests are provided.
Usability	The UI is responsive; English and Vietnamese generation are supported; Swagger documents the backend APIs; PDF/DOCX export is available.

CHAPTER 3. BACKGROUND AND TECHNOLOGIES

3.1 Large Language Models and Text Embeddings

Large Language Models (LLMs) are probabilistic sequence models that estimate the conditional distribution of the next token given a preceding context. Given an input sequence $(x_1, x_2, \ldots, x_t)$, an autoregressive LLM models the probability of a continuation as

$$P(x_{t+1}, \ldots, x_n \mid x_1, \ldots, x_t) = \prod_{i=t+1}^{n} P(x_i \mid x_1, \ldots, x_{i-1}). \tag{3.1}$$

In this project, the LLM is used as a language generation component rather than as a trusted source of factual business knowledge. Its main responsibilities are drafting job descriptions, improving existing job-description content, and generating interview questions. The factual grounding of these outputs is provided by the indexed job-description corpus. In other words, the model is expected to synthesize and verbalize retrieved evidence, while the system controls what contextual information is supplied to it.

Text embeddings provide the numerical representation required for semantic retrieval. An embedding model maps a text segment x to a dense vector

$$\mathbf{e}(x) \in \mathbb{R}^{768}. \tag{3.2}$$

The underlying assumption is that semantically related text segments are mapped to nearby regions of the vector space. Given a query vector $\mathbf{q}$ and a document vector $\mathbf{d}$, their similarity is measured using cosine similarity:

$$\cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\|_2 \|\mathbf{d}\|_2}. \tag{3.3}$$

All vectors in the system are L_2-normalized before indexing. This makes cosine similarity equivalent to the dot product during retrieval, which simplifies ranking and improves numerical consistency. The project uses Gemini embeddings with different task types for document indexing and query embedding, allowing the embedding model to encode the different roles of stored content and search queries [6].

3.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a framework that combines information retrieval with neural text generation. Instead of relying solely on the parametric memory of an LLM, a RAG system retrieves external evidence from a corpus and conditions the generation

process on that evidence. Given a user query q and a document corpus D, the retriever first returns a set of relevant chunks:

$$R(q; D) = \{d_1, d_2, \ldots, d_k\}. \tag{3.4}$$

The generator then produces an output y conditioned on both the original query and the retrieved context:

$$y = G\big(q, R(q; D)\big). \tag{3.5}$$

The main advantage of RAG is that domain knowledge can be updated by modifying the document corpus and rebuilding the retrieval index, without retraining the underlying language model. This is particularly suitable for job-description data, where new roles, technologies, hiring requirements, and company-specific terminology change frequently.

However, the performance of a RAG system depends heavily on several upstream components. Poor chunking may separate related information into different fragments. Low-quality embeddings may fail to capture domain-specific similarity. Noisy or synthetic data may reduce factual reliability. Finally, weak prompt design may cause the LLM to ignore retrieved evidence or over-generalize beyond the provided context. Therefore, this project treats RAG as an end-to-end pipeline whose quality depends on data ingestion, chunking, indexing, retrieval, reranking, and generation.

3.3 Dense, Lexical, and Hybrid Retrieval

The system combines dense retrieval and lexical retrieval because the two methods capture different notions of relevance.

Dense retrieval represents queries and documents as vectors and ranks documents by semantic similarity. This approach is effective when the user query is a paraphrase of the relevant content. For example, a query such as “cloud infrastructure engineer” may retrieve job descriptions containing related phrases such as “platform reliability”, “distributed systems”, or “Kubernetes operations”. Nevertheless, dense retrieval can miss rare exact terms, abbreviations, product names, or role-specific keywords.

Lexical retrieval, in contrast, relies on exact or near-exact term matching. In this project, PostgreSQL full-text search is used through `to_tsvector` and `websearch_to_tsquery`. Job titles are assigned a higher ranking weight, while headings and content are assigned secondary weights. This makes lexical retrieval highly effective for precise terms such as framework names, cloud services, certifications, or seniority labels. Its limitation is that it is less robust to paraphrasing and semantic equivalence.

To benefit from both methods, the project implements hybrid retrieval using Reciprocal Rank Fusion (RRF) [2]. Given multiple ranked lists $\mathcal{R}$, the fused score of a document d is

computed as

$$\mathrm{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + r(d)}, \tag{3.6}$$

where $r(d)$ denotes the rank position of d in ranking r, and k is a smoothing constant. In this project, $k = 60$, which reduces the sensitivity to small differences among top-ranked items while still rewarding documents that appear consistently across retrieval methods.

After fusion, the system applies a diversity-aware selection step. It limits the number of chunks selected from the same job description and penalizes chunks that are too similar to already selected results. This strategy is inspired by Maximal Marginal Relevance (MMR) [3]. The purpose is to avoid returning many near-duplicate chunks from a single JD and to provide the generator with broader evidence.

3.4 HNSW Indexing

For vector search, the project uses Milvus with the Hierarchical Navigable Small World (HNSW) index [4]. HNSW is an approximate nearest-neighbor algorithm based on a multi-layer graph structure. Higher layers provide long-range navigation across the vector space, while lower layers contain denser local neighborhoods. During search, the algorithm greedily traverses the graph from upper layers to lower layers to find vectors close to the query.

The main configuration parameters are M, `efConstruction`, and `ef`. The parameter M controls the maximum number of graph connections per node. A larger M generally improves recall but increases memory usage and index construction time. The parameter `efConstruction` controls the candidate search width during index construction; higher values usually produce a better graph at the cost of slower indexing. At query time, `ef` controls the breadth of search and directly affects the trade-off between latency and recall.

In the current configuration, the system uses $M = 32$ and `efConstruction` $= 200$. At query time, `ef` is selected based on the requested candidate size and is not set below 128. This configuration prioritizes retrieval quality over minimal latency, which is appropriate for the current small-to-medium-scale job-description corpus.

3.5 Authentication and Authorization

The backend uses token-based authentication with JSON Web Tokens (JWTs). After a user submits valid credentials, the system verifies the password hash and creates a signed token containing the user identity, assigned roles, and expiration time. The token is then sent by the frontend in the `Authorization` header using the Bearer scheme.

Authorization is implemented separately from authentication. Protected endpoints declare the roles required to access them, and the `require_roles` dependency checks whether the current user has at least one permitted role. This keeps endpoint logic focused on business

functionality and avoids scattering access-control checks throughout the codebase.

JWT payloads are signed but not encrypted by default [5]. Therefore, the system only stores minimal authorization-related information in the token, such as the subject and role names. Sensitive information, including passwords, password hashes, API keys, and internal configuration values, is never included in the token payload. This design provides a practical balance between stateless authentication, role-based access control, and security hygiene for a development-stage web application.

3.6 Technologies Used

Table 3.1. Main technologies used in the project

Component	Technology	Role
Frontend	React 18, Vite	Single-page application, routing, Markdown editor, preview, and API calls.
Backend	Python, FastAPI	REST API, request validation, dependency injection, Swagger documentation.
Database	PostgreSQL 15	Job descriptions, versions, users, roles, provenance, chunk catalog, and full-text search.
Vector database	Milvus	HNSW/COSINE search over 768-dimensional embedding vectors.
AI models	Gemini	Text generation and embedding generation.
Infrastructure	Docker, ECR-ready image	Local services and cloud deployment preparation.
Testing	pytest, npm build/audit	Backend tests, frontend production build, and dependency audit.

CHAPTER 4. SYSTEM DESIGN AND IMPLEMENTATION

4.1 Overall Architecture

The system follows a layered architecture. The frontend never accesses the database directly; all requests go through FastAPI and authentication dependencies. The service layer coordinates LLM generation, retrieval, versioning, and export. The CRUD/ORM layer encapsulates PostgreSQL access. Milvus is connected lazily so that a temporary vector-store failure does not prevent the backend from starting. Figure 4.1 shows the main implementation components.

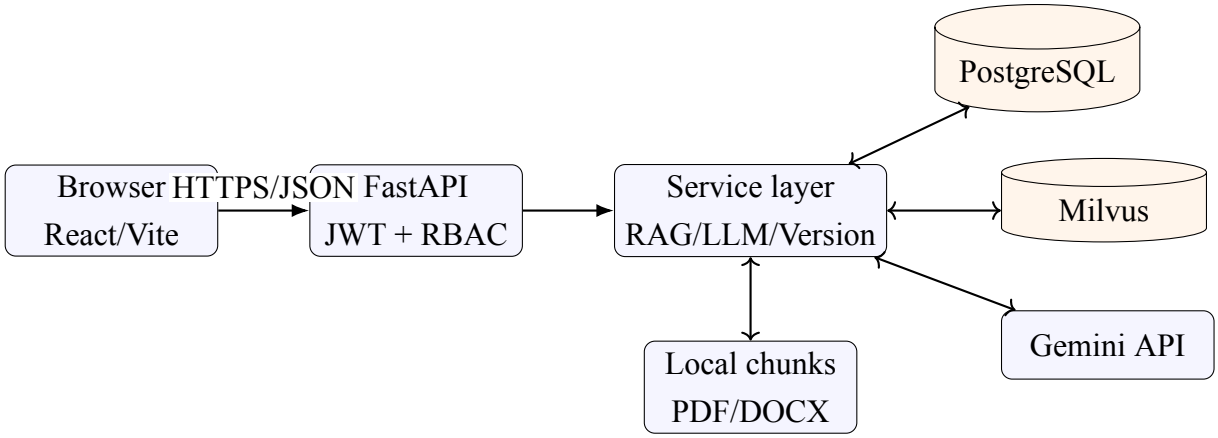


Figure 4.1. Logical architecture of SmartHire Composer

4.2 Backend Design

The backend is organized into five main groups. The `api` package defines routes and dependencies. The `schemas` package defines request and response objects with Pydantic. The `crud` package contains database operations. The `services` package implements business logic such as LLM orchestration, retrieval, version management, and interview-question generation. The `utils` package provides auxiliary functions such as PDF/DOCX export.

This organization keeps endpoint functions concise and avoids mixing HTTP logic with database operations or LLM orchestration. It also allows future developers to replace a component, such as the embedding provider or vector database, without rewriting the entire API layer.

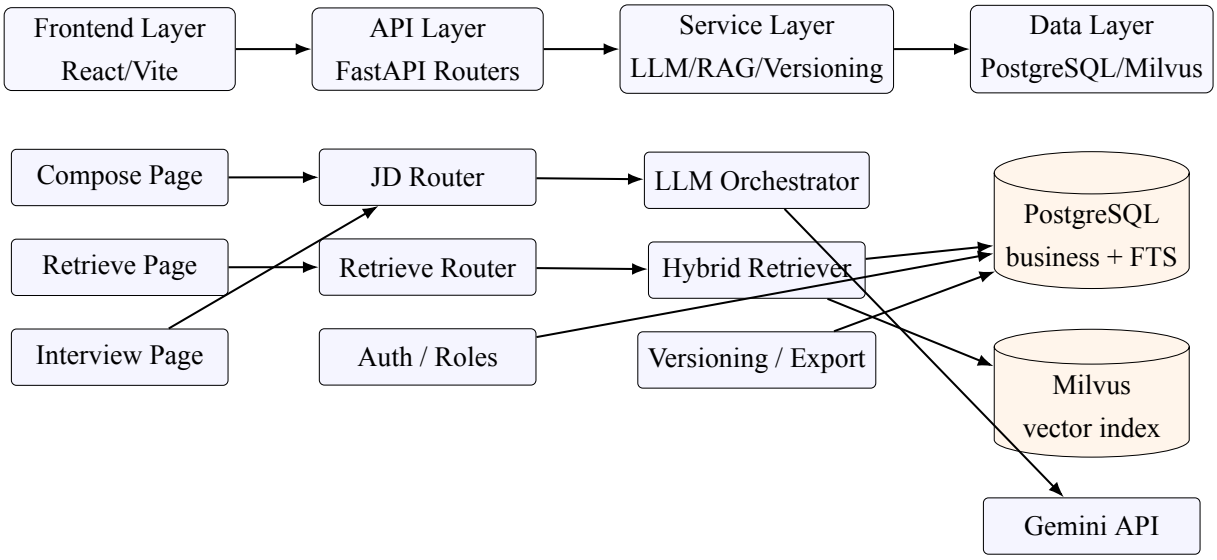


Figure 4.2. UML component diagram of the main software modules

4.3 Data Model

PostgreSQL contains two groups of data. The business group includes `users`, `roles`, `user_roles`, `job_descriptions`, `jd_versions`, and taxonomy-related tables. The RAG group includes `jd_chunks` and `rag_index_metadata`. The `jd_chunks` table stores the same `chunk_id` used in Milvus, along with chunk text, heading, hash, path, and token estimate. This allows lexical retrieval and dense retrieval to be merged using a stable key. The manifest records provider, model, dimension, chunker version, and chunk count. Before dense retrieval, the runtime configuration is checked against the manifest to prevent mixing incompatible vector spaces.

Table 4.1. Core database tables

Table	Content
`job_descriptions`	Current job description, recruitment metadata, provenance, source hash, and version number.
`jd_versions`	Markdown snapshots by version, editor, timestamp, and change summary.
`jd_chunks`	Chunk text, heading, hash, file path, length, and job-description link.
`rag_index_metadata`	Embedding model, vector dimension, chunker version, document count, and chunk count.
`users`, `roles`, `user_roles`	Account and many-to-many role mapping for RBAC.
`job_families`, `tags`	Job-family taxonomy and content labels.

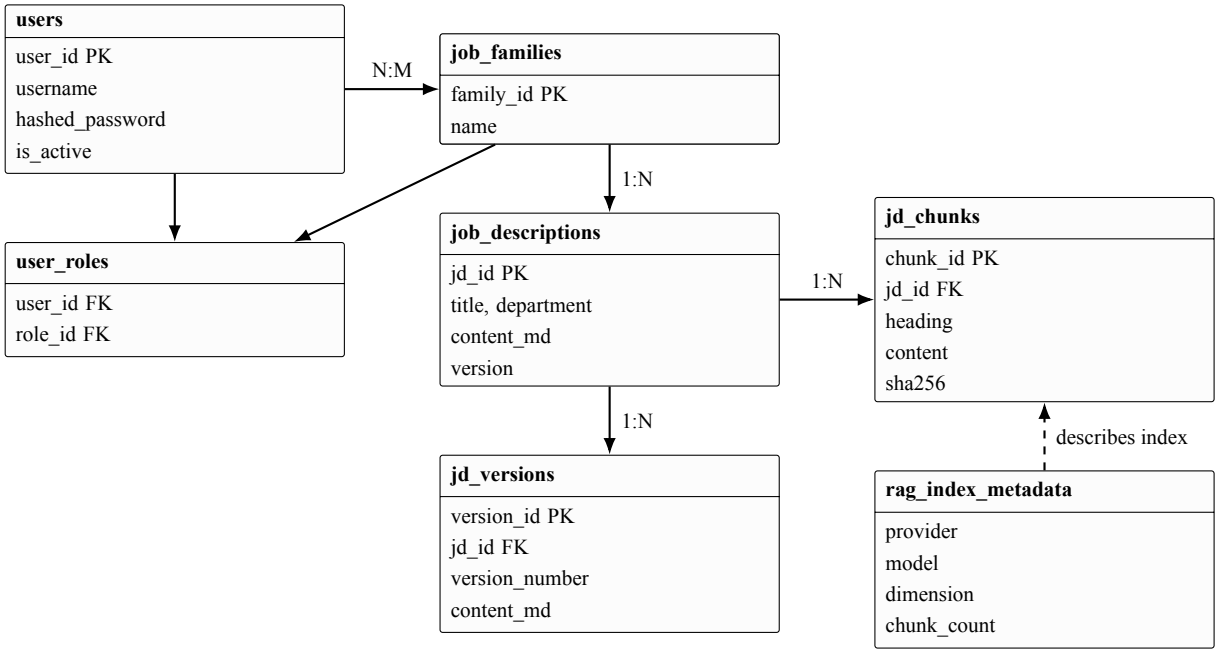


Figure 4.3. Entity-relationship overview of the core database schema

4.4 Real Job Data Ingestion

The crawler supports Greenhouse Job Board API, Lever Postings API, and JSON-LD `JobPosting`. Each adapter converts source-specific fields into a common structure containing external ID, company, title, description, URL, department, location, employment type, and posting time. The normalized content is stored as Markdown with front matter. SHA-256 hashes are used to detect changes. The ETL process upserts by `source_name` and `source_external_id`; when the hash does not change, the system updates the observed timestamp instead of creating redundant versions.

The data pipeline follows conservative data-use principles. It collects only public job-posting information through allowed APIs or public pages, uses a clear user agent, respects delay settings, and preserves attribution. It does not access private APIs, bypass CAPTCHA, or collect candidate information. Old synthetic job descriptions are separated from the production-like corpus; the reported experiment uses only 75 real public job descriptions.

4.5 RAG Indexing Pipeline

The chunker prioritizes heading, paragraph, and sentence boundaries. Long blocks are hard-split at word boundaries with overlap. The maximum chunk size is 1,200 characters, and the reported average is 1,007 characters. Embeddings are generated before the index is switched, so quota errors do not leave a partially built index in use. Gemini is called in batches, and the script sleeps between batches according to free-tier limits. After insertion, the Milvus collection is compacted; the PostgreSQL catalog and manifest are updated only after the dense index

succeeds.

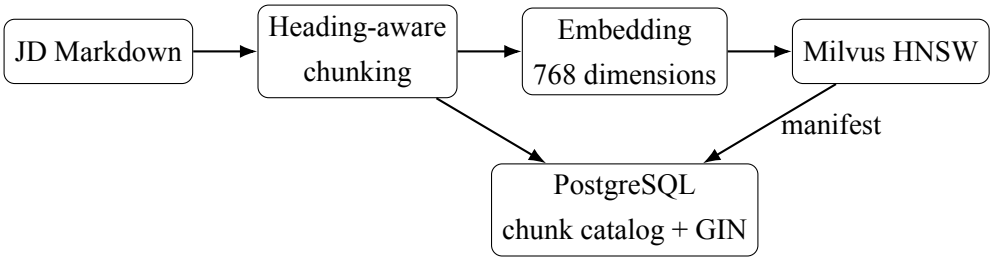


Figure 4.4. RAG indexing pipeline

4.6 Hybrid Retrieval Algorithm

For each query, the system retrieves up to 40 dense candidates and 40 lexical candidates. Dense search can filter valid job-description IDs by company or department before sending an expression to Milvus. Lexical search assigns high weight to title and uses a GIN index on text content. The two lists are merged with Equation 3.6. The final selection chooses the candidate with the highest effective score:

$$s'(c) = s_{RRF}(c) - 0.08 \max_{d \in S} J\big(T(c), T(d)\big), \tag{4.1}$$

where J is Jaccard similarity over token sets and S is the set of already selected results. By default, no more than two chunks are selected from the same job description, and only one chunk per job description is used when constructing a generation prompt. If embedding fails in hybrid mode, the endpoint degrades to lexical retrieval instead of failing the whole request.

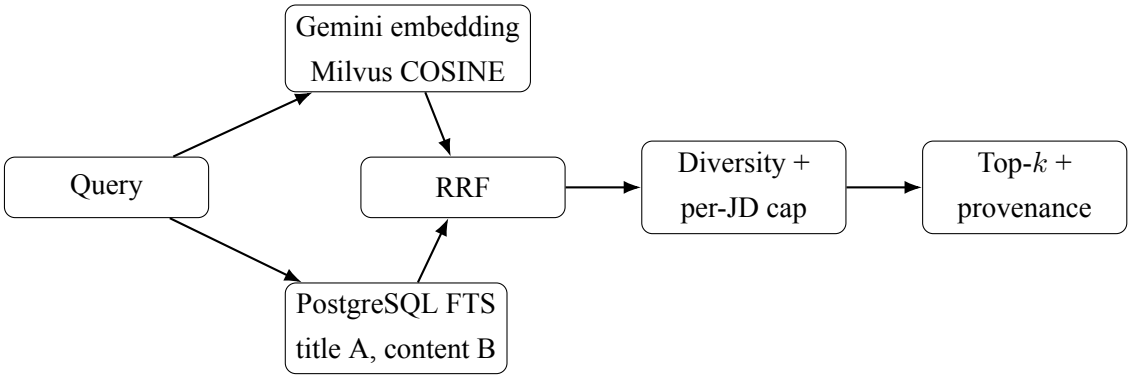


Figure 4.5. Hybrid retrieval flow

4.7 LLM Orchestration and Safety

External job-description content is wrapped in an `UNTRUSTED_SOURCE` block. The prompt instructs the model to extract job-related facts only and to ignore any instruction-like text from the source. The system asks the model to synthesize rather than copy long passages

and to keep source URLs visible where appropriate. If the LLM returns a quota or service error, JD generation returns a minimal deterministic template, improvement keeps the original content, and interview-question generation uses an offline fallback with a deterministic seed.

4.8 Frontend Design

The frontend includes Dashboard, Compose, Interview, Retrieve, and Roles pages. The Compose page is the central working area. It contains a metadata form, a Markdown editor, a sanitized preview using DOMPurify, version history, export actions, and an AI Suggestions panel. The Retrieve page allows users to select Hybrid, Dense, or Lexical retrieval and displays score, method, company, title, URL, chunk path, and snippet. The interface supports both English and Vietnamese generation through a language selector.

4.9 Implemented Product Interface

This section presents the actual implemented user interface of SmartHire Composer. The screenshots are included to complement the UML and architectural diagrams: while the diagrams explain the design at an abstract level, the product screenshots demonstrate that the main user workflows have been realized in the web application. The interface follows a dark neumorphic visual style, uses a persistent navigation sidebar, and groups related actions into cards to reduce cognitive load during job-description authoring and retrieval.

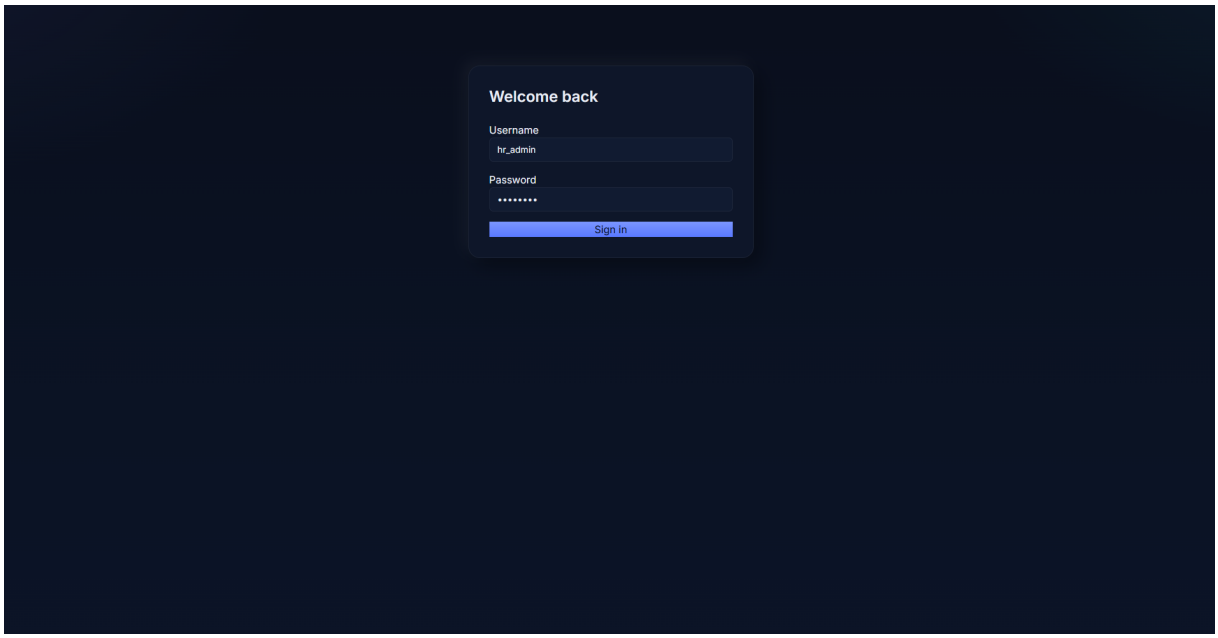


Figure 4.6. Login screen for authenticated access to the SmartHire Composer workspace

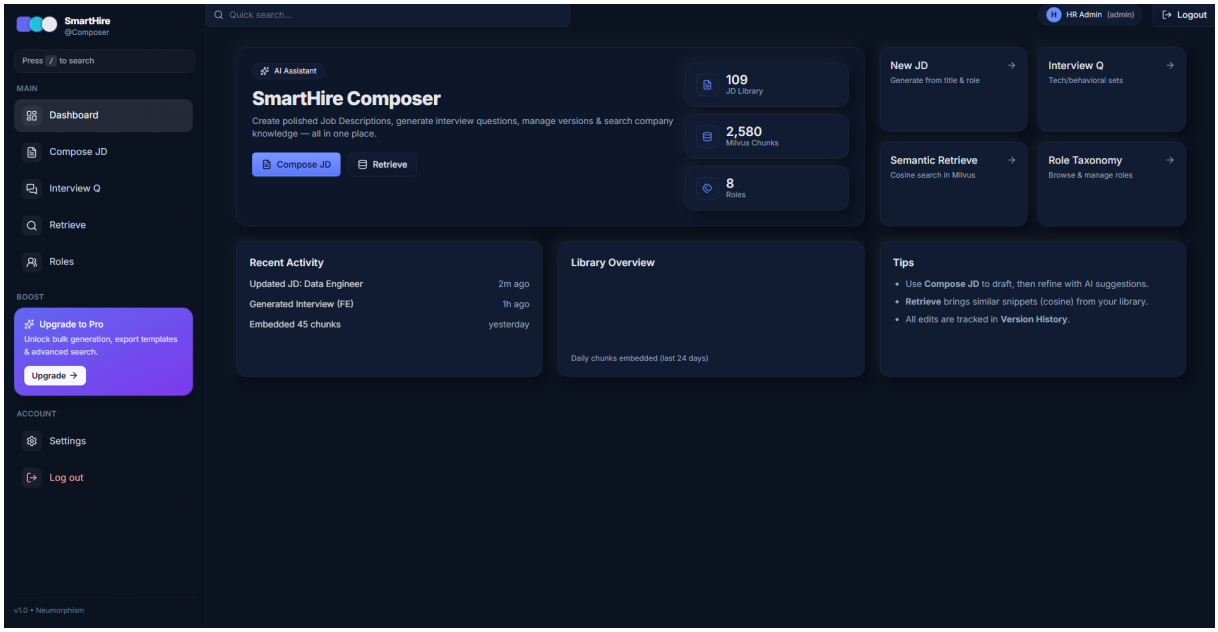


Figure 4.7. Dashboard screen showing workspace shortcuts, library statistics, recent activity, and navigation to major modules

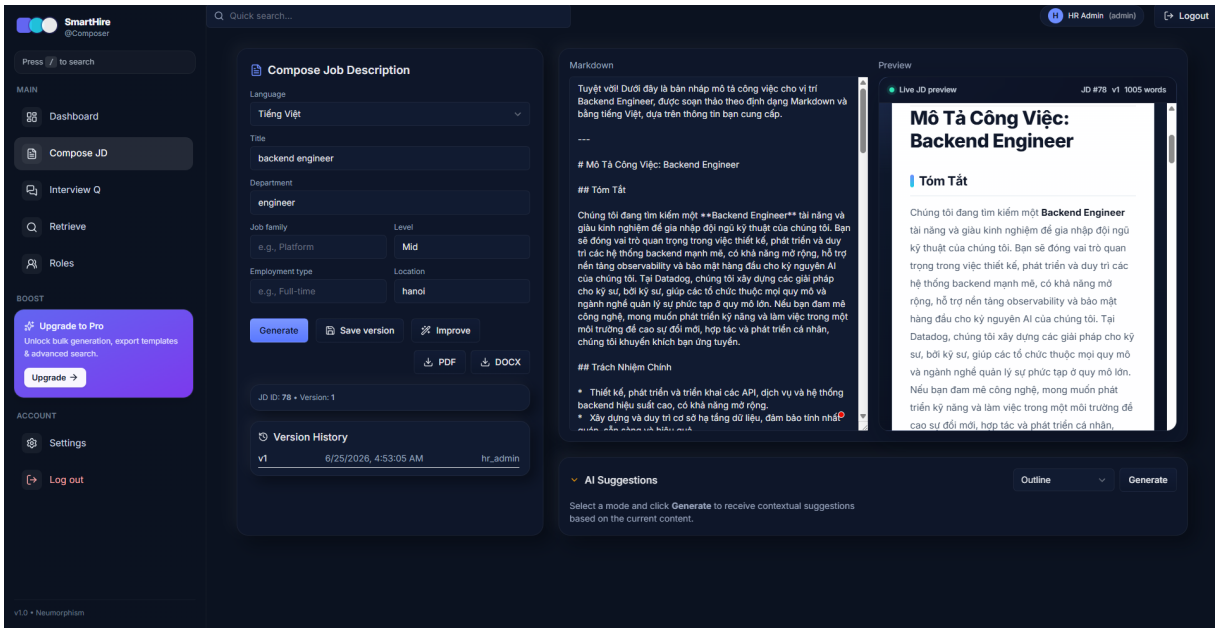
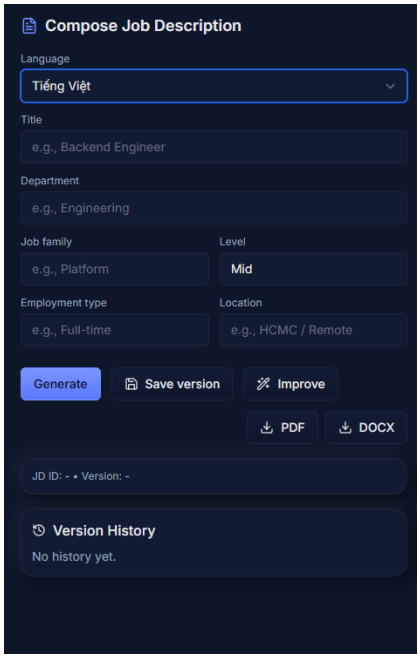
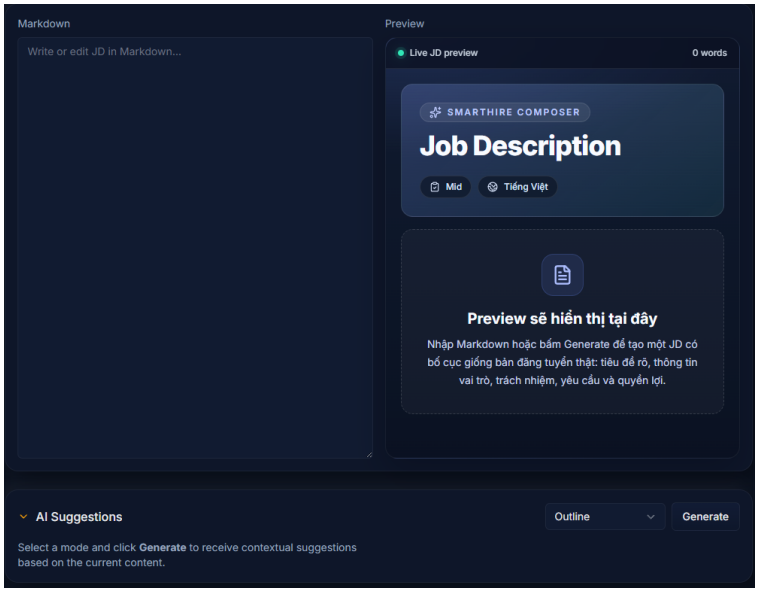


Figure 4.8. Compose JD screen after generation, including metadata input, Markdown editor, live preview, version history, export actions, and AI Suggestions



(a) Metadata form and action buttons



(b) Empty Markdown editor, preview placeholder, and AI Suggestions panel

Figure 4.9. Detailed views of the Compose JD interface before generation

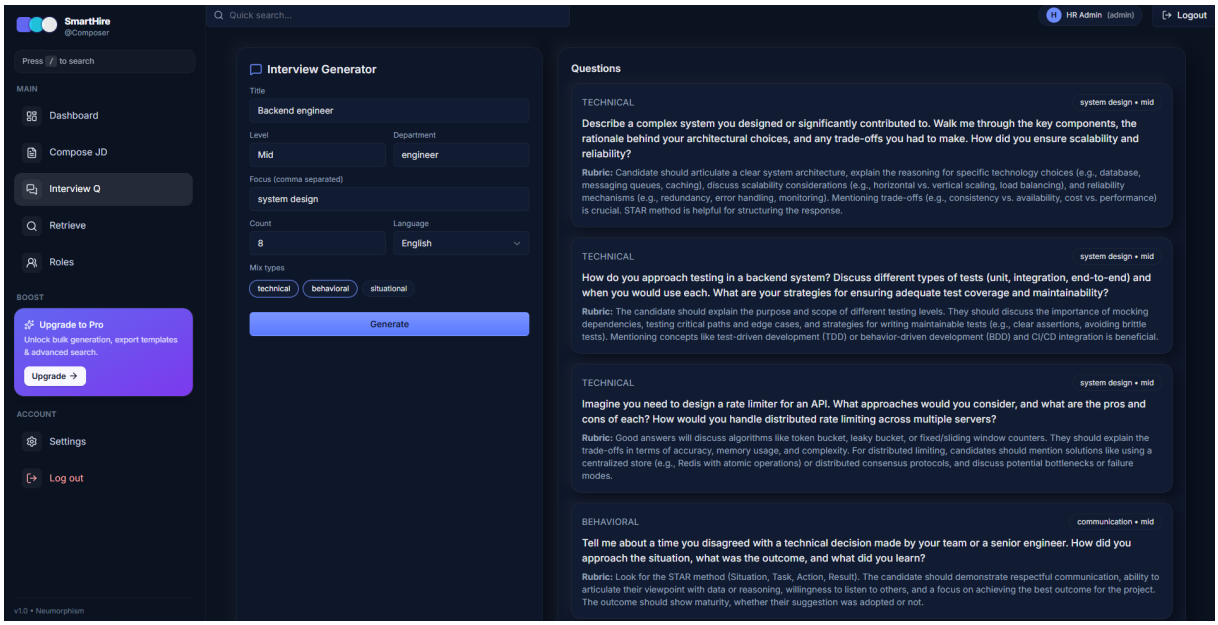
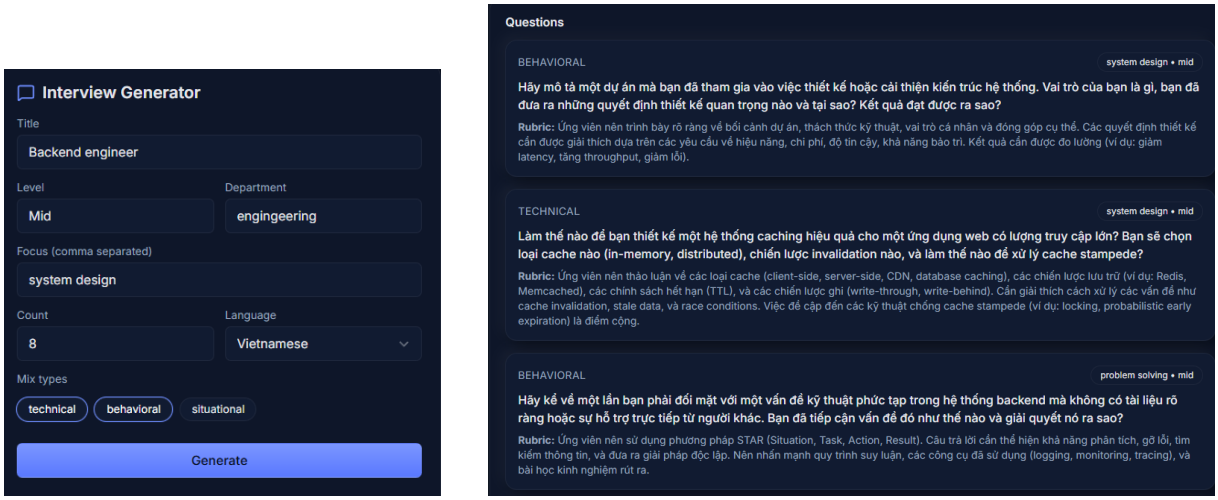


Figure 4.10. Interview Generator screen with input parameters and generated technical, behavioral, and situational questions



(a) Interview generation form **(b)** Generated question cards with rubric text

Figure 4.11. Detailed views of the Interview Generator module

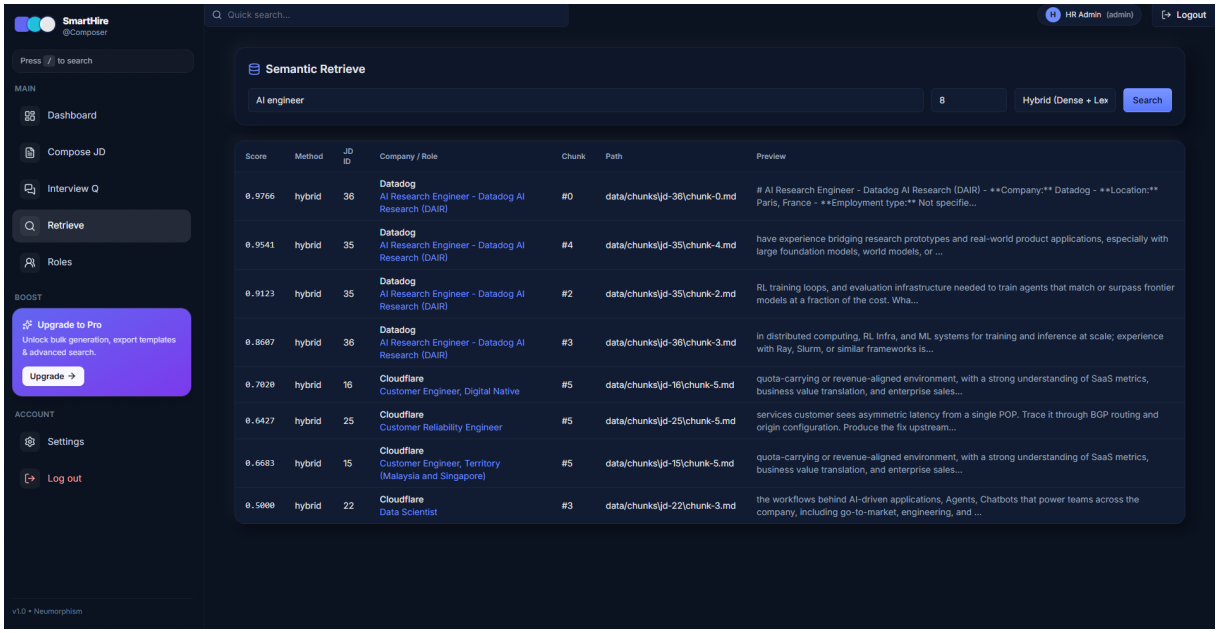


Figure 4.12. Semantic Retrieve screen showing hybrid retrieval results with scores, method, source job, chunk index, file path, and snippet preview

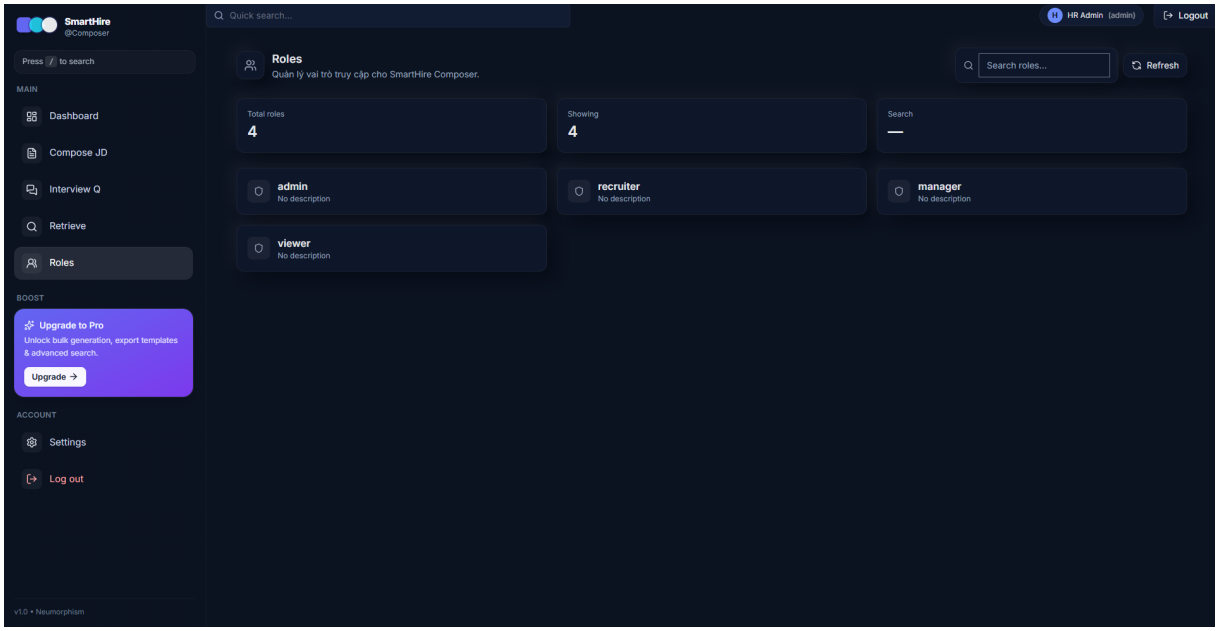


Figure 4.13. Roles screen showing the available role-based access-control groups used by the application

The implemented interface supports the same functional boundaries described in the use-case model: authentication is separated from the workspace, the dashboard provides a high-level overview, Compose JD handles the main authoring workflow, Interview Generator covers question generation, Semantic Retrieve exposes the RAG search layer, and Roles presents the RBAC configuration. These screenshots therefore serve as visual evidence that the designed modules were translated into concrete product screens.

4.10 Deployment Preparation

The backend is containerized with Docker and can be pushed to AWS Elastic Container Registry. In local development, environment variables are loaded from a `.env` file. For cloud deployment, secrets such as database credentials, JWT secret, and Gemini API key should be stored in AWS Systems Manager Parameter Store or AWS Secrets Manager and injected into ECS Task Definitions or EKS Deployments. This avoids hard-coding secrets in source code or container images.

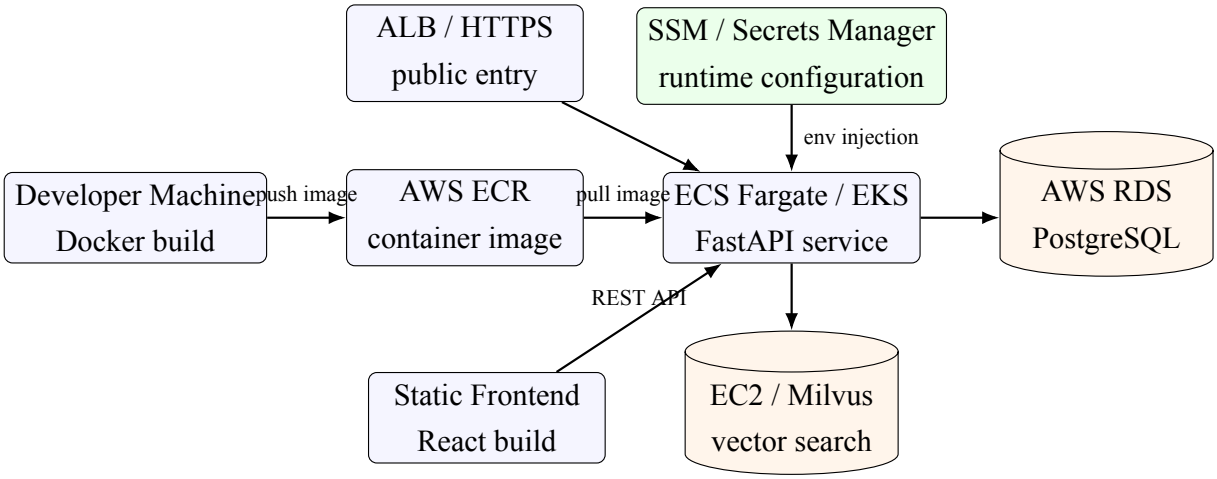


Figure 4.14. Cloud deployment view with container registry and managed secrets

CHAPTER 5. IMPLEMENTATION, TESTING, AND EVALUATION

5.1 Experimental Environment

The experiments were conducted in a development environment using Python 3.12, Node.js, and Docker Desktop. PostgreSQL 15, Milvus, etcd, and internal object storage components were run with Docker Compose. The backend and frontend were run directly during development to simplify debugging. The embedding model was `gemini-embedding-001`, the vector dimension was 768, and the Milvus collection used HNSW with cosine similarity.

Table 5.1. Experimental corpus statistics

Metric	Value
Number of companies	3
Number of public JDs	75
Cloudflare / Datadog / Figma JDs	25 / 25 / 25
Number of chunks	589
Average chunk length	1,007 characters
Maximum chunk length	1,200 characters
Vector dimension	768

5.2 Retrieval Evaluation Method

The `evaluate_rag.py` script selected the first 15 real job descriptions and used each title as a query. The original job description was treated as the relevant document. For a top-k result list, Recall@k measures the percentage of queries for which the relevant document appears in the top-k results. Mean Reciprocal Rank is defined as

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}. \tag{5.1}$$

The unique JD ratio measures the number of distinct job descriptions divided by the number of returned results. The benchmark was run with at most one chunk per job description in the final top-5 list in order to measure document coverage.

5.3 Retrieval Results

Table 5.2. Results on 15 self-retrieval queries, top-$k = 5$

Method	Recall@5	MRR	Unique JD ratio
Lexical after hyphen normalization	1.0000	1.0000	1.0000
Dense	1.0000	0.9556	1.0000
Hybrid RRF	1.0000	0.9556	1.0000

Before diversity control, the observed top-10 results of several queries contained only 8 or 9 unique job descriptions because one job could occupy multiple positions. After applying the per-JD cap, the top-5 benchmark achieved a unique ratio of 1.0. Lexical retrieval initially missed titles containing hyphens because `websearch_to_tsquery` interpreted the hyphen as a negation operator. Normalizing separator characters improved lexical Recall@5 and MRR from 0.8 to 1.0. This result shows the value of error analysis beyond simply changing models.

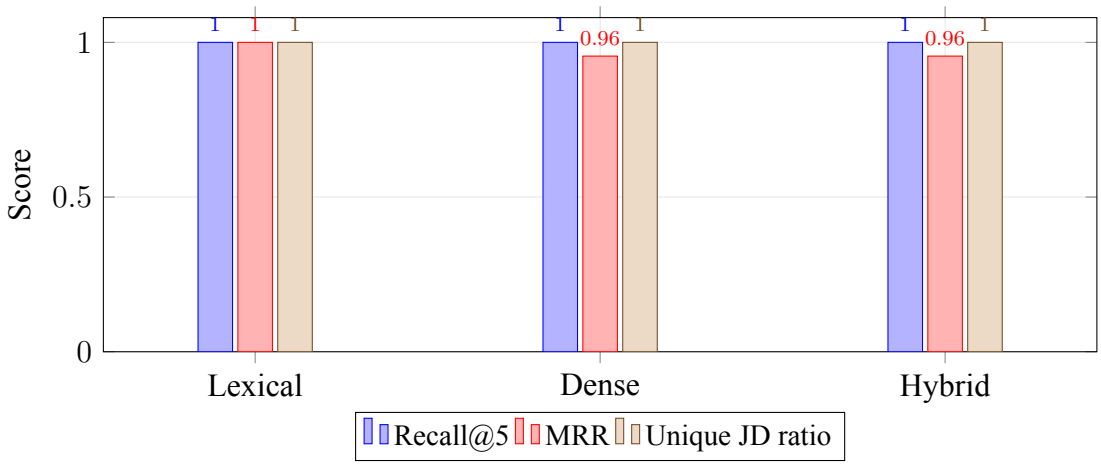


Figure 5.1. Retrieval evaluation summary on the 15-query self-retrieval benchmark

Dense and hybrid retrieval had lower MRR than lexical retrieval on this title-based self-retrieval benchmark because the title is an almost perfect keyword signal. This does not prove that lexical retrieval is better for natural user queries. Dense and hybrid retrieval are expected to be more useful when users describe a role by responsibilities, skills, or business intent rather than by the exact title. An expert-labeled benchmark is needed to verify that expectation.

5.4 Software Testing

Table 5.3. Software testing results

Group	Content	Result
Chunking	Empty input, long block, size limit, and content preservation	Passed
Milvus schema	Collection and required fields	Passed
Lexical retrieval	Relevance and diversity	Passed
Metadata filter	Company-based filtering	Passed
Backend E2E	Login, /auth/me, versioning, export, retrieval, and interview fallback	Passed
Frontend	Vite production build	Passed
Dependency audit	npm audit	0 vulnerabilities

In the final local measurement, the automated backend tests reported `5 passed`. The frontend production build completed successfully, and npm audit reported no known vulnerabilities. PostgreSQL full-text search used a Bitmap Index Scan on the GIN index, and a sample query such as “sales account executive” completed in sub-millisecond time in the local environment.

5.5 Discussion and Validity Threats

The corpus has real provenance but remains small and includes only three technology companies. Therefore, the distribution of job titles, languages, and locations is not representative of the entire labor market. The self-retrieval benchmark using titles is biased toward lexical search and does not measure relevance from the perspective of HR experts. High recall also does not directly imply high quality of the generated job descriptions. Furthermore, the LLM is an external service subject to quota limits and temporary service errors. The fallback mechanisms preserve system behavior, but their output quality is lower than full LLM generation.

To reduce these threats, the system records manifest information, timestamps, source URLs, and reproducible benchmark scripts. The reported results should therefore be interpreted as evidence that the prototype is operational and testable, not as a general conclusion about all recruitment domains.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project developed SmartHire Composer, a RAG-enhanced web system for job-description authoring and interview-question generation. The system supports authentication and RBAC, job-description generation, Markdown editing and preview, version history, PDF/DOCX export, interview-question generation, public job-data ingestion with provenance, vector indexing, and dense/lexical/hybrid retrieval. From a technical perspective, the system connects PostgreSQL and Milvus through stable chunk identifiers, checks embedding compatibility through a manifest, improves retrieval diversity, and applies prompt-safety measures when using external source text.

The experimental corpus contains 75 real public job descriptions and 589 chunks. On a 15-query self-retrieval benchmark, all three retrieval modes achieved Recall@5 of 1.0 after the lexical normalization and diversity improvements. Five automated backend tests passed, and the frontend build completed successfully. The most important result of Project 1 is not only a user interface that calls an LLM, but a traceable and testable data pipeline from source ingestion to retrieval and generation.

6.2 Limitations

The current system still has several limitations. It does not yet include expert-labeled relevance judgments, a rubric-based evaluation of generated job descriptions, a production scheduler for crawlers, fully incremental vector updates, automatic detection of closed job postings, multi-tenant data isolation, cloud secret management, HTTPS termination, distributed tracing, or large-scale load testing. The local lexical fallback is based on hashing and should not be considered equivalent to semantic embeddings.

6.3 Future Work

Future development can proceed in four directions. First, the project should build an expert-labeled evaluation set with natural-language HR queries, relevance judgments, and a generation-quality rubric. Second, the retrieval pipeline can be improved with a cross-encoder reranker, query expansion, and learning-to-rank, followed by A/B testing of RRF weights. Third, the data pipeline should support scheduled crawling, hash-based incremental indexing, closed-posting detection, and retention policies. Fourth, production readiness should be improved with HTTPS, secret management, OpenTelemetry, Redis caching, worker queues, PostgreSQL backup, and load testing. In the recruitment domain, future versions should also include bias

checking, inclusive-language review, and human-in-the-loop approval before generated content is published.

CHAPTER 7. SOURCE CODE

Link github: https://github.com/QuanNguyen28/Project1.git

BIBLIOGRAPHY

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[2] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," *Proceedings of SIGIR*, 2009.

[3] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," *Proceedings of SIGIR*, 1998.

[4] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, 2020.

[5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015. https://www.rfc-editor.org/rfc/rfc7519

[6] Google, "Gemini Embeddings," Google AI for Developers. https://ai.google.dev/gemini-api/docs/embeddings, accessed June 2026.

[7] Milvus, "Milvus Documentation: Vector Search and HNSW Index." https://milvus.io/docs, accessed June 2026.

[8] PostgreSQL Global Development Group, "PostgreSQL Full Text Search." https://www.postgresql.org/docs/15/textsearch.html, accessed June 2026.

[9] FastAPI, "FastAPI Documentation." https://fastapi.tiangolo.com/, accessed June 2026.

[10] Greenhouse Software, "Job Board API." https://developers.greenhouse.io/job-board.html, accessed June 2026.

[11] Lever, "Lever Postings API." https://github.com/lever/postings-api, accessed June 2026.

[12] Schema.org, "JobPosting." https://schema.org/JobPosting, accessed June 2026.

CHAPTER A. INSTALLATION AND OPERATION GUIDE

A.1 Environment Setup

Listing A.1. Environment setup commands on Windows PowerShell

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
cd frontend
npm ci
cd ..
.\scripts\bootstrap.ps1
```

A.2 Running the Application

Listing A.2. Running backend and frontend

```
# Backend
$env:PYTHONUTF8="1"
.\.venv\Scripts\python.exe -m uvicorn src.main:app --host 0.0.0.0
   --port 8000

# Frontend
cd frontend
npm run dev -- --host 0.0.0.0
```

A.3 Crawling, ETL, and Indexing

Listing A.3. Data ingestion and indexing commands

```
.\.venv\Scripts\python.exe scripts\crawl_jobs.py ^
  --source greenhouse --site cloudflare ^
  --company Cloudflare --limit 25

.\.venv\Scripts\python.exe etl\jd_etl.py ^
  --dir data\real_jd --author public-api-ingestion

.\.venv\Scripts\python.exe embeddings\jd_chunk_embed.py
```

CHAPTER B. MAIN API LIST

Method	Endpoint	Purpose
POST	`/auth/token`	Log in and receive a JWT.
GET	`/auth/me`	Retrieve the current user profile.
POST	`/v1/jd/generate`	Generate a job description, optionally with RAG context.
PUT	`/v1/jd/update`	Save a new version of a job description.
GET	`../version-history/{id}`	List versions of a job description.
POST	`/v1/jd/improve`	Improve an existing job description.
POST	`/v1/jd/suggest`	Suggest content for a selected JD section.
GET	`/v1/jd/export/{id}`	Export a JD to PDF or DOCX.
POST	`/v1/interview/generate`	Generate interview questions.
POST	`/v1/retrieve`	Hybrid, dense, or lexical retrieval.
POST	`/v1/retrieve/similar`	Dense retrieval compatible with the older API.
GET	`/health/live`	Liveness check.
GET	`/health/ready`	Readiness check for storage and index.

CHAPTER C. SOURCE CODE STRUCTURE

The project is organized into clearly separated modules, following a layered architecture between the backend, frontend, data ingestion pipeline, RAG components, infrastructure scripts, and documentation. This structure improves maintainability, testing, and future extension.

Table C.1. Main source-code organization

Directory	Description
src/	FastAPI backend, including API routes, service layer, ORM models, schemas, authentication, and business logic.
frontend/	React/Vite frontend for job description composition, preview, retrieval, interview question generation, and user interaction.
embeddings/	RAG-related processing, including text chunking, embedding generation, and vector index construction.
etl/	ETL pipeline for importing Markdown-based job descriptions into PostgreSQL.
scripts/	Utility scripts for crawling real job postings, seeding data, evaluating retrieval quality, and running project workflows.
./migrations/	PostgreSQL migration files used to create and evolve the database schema.
data/real_jd/	Real public job descriptions collected from permitted sources, stored with provenance metadata.
data/chunks/	Local Markdown chunks generated from job descriptions for retrieval and indexing.
tests/	Automated tests for core backend and retrieval functionality.
report/	LaTeX source files and generated report artifacts.

CHAPTER D. REPRESENTATIVE TEST CASES

ID	Precondition / Data	Action	Expected Result
T01	User `hr_admin` is seeded	POST `/auth/token`	A valid Bearer JWT is returned.
T02	Recruiter JWT exists	Create a new JD	JD and version 1 are saved.
T03	A JD exists	Update twice	Version numbers increase sequentially, history is ordered from newest to oldest.
T04	Milvus and catalog are synchronized	Query “sales account executive”	Top-5 results are relevant and do not repeat the same JD.
T05	Gemini embedding fails temporarily	Hybrid retrieval	The system falls back to lexical retrieval without breaking the API.
T06	Company filter = Figma	Retrieve with filter	All returned results belong to Figma.
T07	Content contains special characters	Export PDF/DOCX	HTTP 200 is returned and the file can be opened.
T08	Milvus collection is missing	GET readiness	HTTP 503 is returned with index check = false.